

Note :- As a developer we are responsible to handle the Exception. System admin is responsible to handle the Error because we cannot recover from error.

An error represents serious problem.

WAP to show that Exception is the super class of all types of Exception whether it is checked Or unchecked :

Exception format :

Whenever we print any exception object by using IO.println() then It will invoke toString() method of Throwable class because Throwable class has overridden toString() method from Object class which return the following details in String format :

```
public String toString()
{
    String s = getClass().getName();
    String message = getLocalizedMessage();
    return (message != null) ? (s + ": " + message) : s;
}

Format is : Fully Qualified Name: errorMessage

Package Name + Class Name : errorMessage

This logic is written in the toString() method of Throwable class.

//Program .
package com.ravi.basic;

public class ExceptionAsSuper
{
    public static void main(String[] args)
    {
        Exception e1 = new ArithmeticException();
        IO.println(e1); //java.lang.ArithmeticException

        Exception e2 = new ArithmeticException("Dividing a number by zero");
        IO.println(e2); //java.lang.ArithmeticException: Dividing a number by zero

        Exception e3 = new NullPointerException("Ref is pointing to null");
        IO.println(e3); //java.lang.NullPointerException: Ref is pointing to null
    }
}
```

WAP that describes, whenever an exception is encountered in the program then our program will be terminated abnormally (**halt in the middle**)

AbnormalTermination.java

```
void main()
{
    IO.println("Hello Learners!!!");
    IO.println(10/0);
    IO.println("Thank you");
}
```

Note : In the above program we are getting the ArithmeticException so, program will be terminated in the middle (No try-catch), IT IS CALLED **ABNORMAL TERMINATION**.

Now, ArithmeticException object will be propagated (will be transferred) to the caller method i.e. JVM, JVM has default exception handler, which will print the exception details without handling the exception.

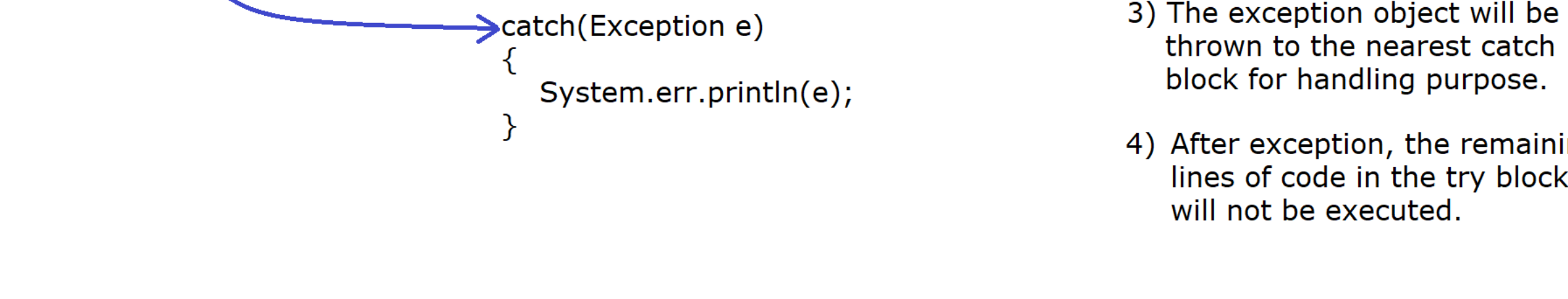
In order to handle the exception and to work with exception. java software people has provided the following keywords :

- 1) try block
- 2) catch block
- 3) finally block (try with resources JDK 1.7V)
- 4) throw
- 5) throws

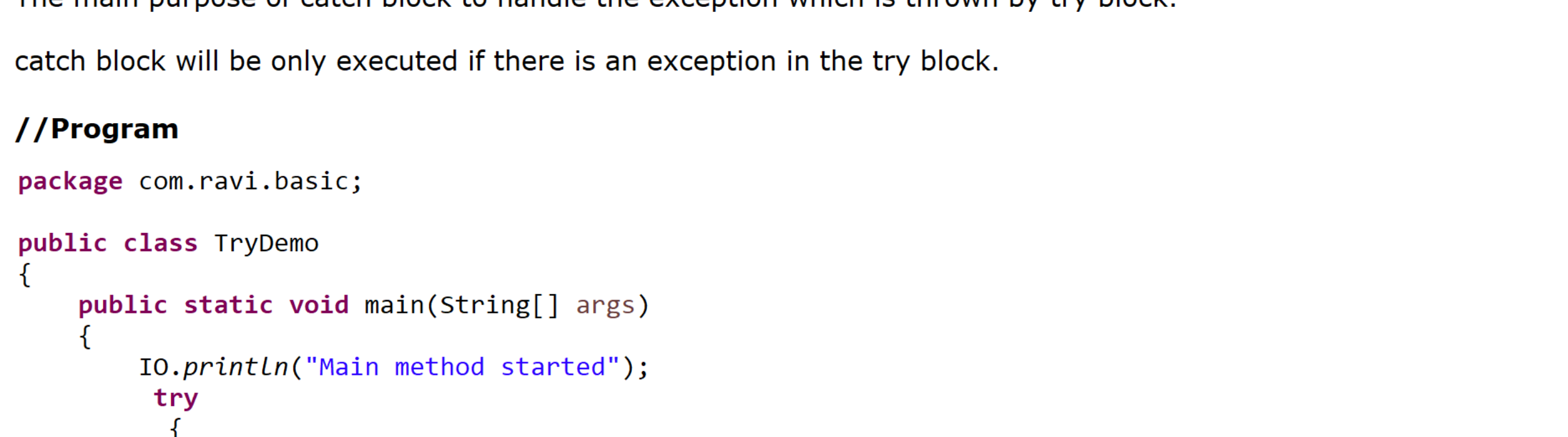
try block :

* Whenever our statement is "**Error suspecting statement**" OR "**Risky Statement**" then we should write that statement inside the try block.

* try block we cannot write independently, It must be followed by either catch block OR finally block OR both.



* We cannot write any kind of statement in b/w try-catch block.



Work of try-catch block

- 1) Will trace the code line by line
- 2) If any exception is encountered then with the help of JVM, appropriate exception object will be created.
- 3) The exception object will be thrown to the nearest catch block for handling purpose.
- 4) After exception, the remaining lines of code in the try block will not be executed.

catch block :

The main purpose of catch block to handle the exception which is thrown by try block.

catch block will be only executed if there is an exception in the try block.

```
//Program
package com.ravi.basic;

public class TryDemo
{
    public static void main(String[] args)
    {
        IO.println("Main method started");
        try
        {
            Integer x = Integer.valueOf(IO.readLine("Enter the value of x :"));
            Integer y = Integer.valueOf(IO.readLine("Enter the value of y :"));
            Integer z = x/y;
            IO.println("z value is :"+z);
            IO.println("End of try block");
        }
        catch(Exception e)
        {
            IO.println("Inside Catch Block");
            System.err.println(e);
        }
        IO.println("Main method ended");
    }
}
```